could produce erroneous or inconsistent outcomes. Internally, the **chunkById** method will always return models whose last model in the chunk's previous chunk has an **id** column larger than zero:

```
Office::where('closed', true)
  ->chunkById(200, function (Collection $offices) {
   $offices->each->update(['closed' => false]);
  }, $column = 'id');
```

Chunk Using Lazy Collections

In that it conducts the query in pieces behind the scenes, the **lazy** method functions similarly to the **chunk** method. However, the **lazy** method returns a flattened **LazyCollection** of Eloquent models, allowing you to interact with the results as a single stream rather than simply handing each chunk into a callback as is:

```
use App\Models\Office;
foreach (Office::lazy() as $office) {
  // Code Logic
}
```

Use the **lazyById** method if you want to filter the outcomes of the **lazy** method based on a column that will also be updated as you iterate over the outcomes. Internally, the **lazyById** method will always return models whose **id** columns are bigger than the ones of the last model in the **chunk** before:

```
Office::where('closed', true)
  ->lazyById(200, $column = 'id')
  ->each->update(['closed' => false]);
```

The **lazyByIdDesc** method allows you to filter the results based on the **id's** descending order.

Cursors

The **cursor** method can be used to drastically reduce your application's memory consumption when iterating through tens of thousands of Eloquent model records, similar to the **lazy** method.

Individual Eloquent models won't be hydrated until they are iterated through; the **cursor** technique will only run one database query. As a result,